

ORGANIZAÇÃO DO DESENVOLVIMENTO

Divisão de Trabalho para 2 Desenvolvedores

Estratégia de execução paralela para a loja virtual, considerando a estrutura inicial do projeto já concluída.

ESTRATÉGIA RECOMENDADA

Divisão por fluxos verticais

Cada desenvolvedor assume um fluxo completo, incluindo entidades, persistência, interface e testes. Isso reduz dependências, bloqueios e conflitos de integração.

DESENVOLVEDOR 1

Catálogo e administração

Produtos, descoberta e gestão do catálogo.

ESCOPO PRINCIPAL

- Product, Category e Tag.
- Repositórios e dados iniciais do catálogo.
- Página inicial, catálogo e detalhes do produto.
- Busca, filtros e ordenação.
- CRUD administrativo de produtos e categorias.
- Controle de estoque e produtos ativos.
- Testes do catálogo e do CRUD.

DESENVOLVEDOR 2

Compra e cliente

Carrinho, checkout e acompanhamento da compra.

ESCOPO PRINCIPAL

- User, Address, Cart e CartItem.
- Order, OrderItem, Payment e Delivery.
- Repositórios de usuário, carrinho e pedidos.
- Carrinho e checkout.
- Pagamento e frete simulados.
- Login, área do cliente e downloads.
- Administração de pedidos e clientes.

Fluxos independentes

DEV 1

Catálogo → Produtos → Estoque →
Administração do catálogo

DEV 2

Cliente → Carrinho → Checkout → Pedidos

Preparação conjunta antes da divisão

Os contratos compartilhados devem ser definidos e integrados à branch principal antes das implementações paralelas.

- Estrutura das entidades.
- IDs e relacionamentos.
- Interfaces dos repositórios.
- Chaves do localStorage.
- Status de pedido e pagamento.
- Formato dos dados iniciais.
- Regras de estoque.
- Contrato produto–carrinho–pedido.

EXECUÇÃO

Rodadas de trabalho paralelo

Cada rodada entrega partes integráveis e reduz o tempo entre implementação e revisão.

1

DEV 1

Domínio e catálogo

Product, Category, Tag, repositórios, seed, catálogo e detalhes do produto.

DEV 2

Domínio e conta

User, Address, Cart, repositórios, estado do carrinho, login e área básica do cliente.

2

DEV 1

Administração do catálogo

Busca, filtros, ordenação, CRUD de produtos, categorias e estoque.

DEV 2

Compra

Pedido, pagamento, entrega, checkout, frete simulado e geração do pedido.

3

DEV 1

Dashboard

Indicadores administrativos, produtos com estoque baixo e refinamento da gestão.

DEV 2

Pós-compra

Histórico, detalhes, downloads e administração de pedidos e clientes.

4

DEV 1

Qualidade visual

Responsividade, acessibilidade e revisão dos fluxos de catálogo e administração.

DEV 2

Qualidade funcional

Testes de carrinho, checkout, pedidos, persistência e cenários de estoque.

ORGANIZAÇÃO DO CÓDIGO

Propriedade principal dos arquivos

A propriedade indica quem conduz as alterações. Revisões cruzadas continuam recomendadas.

DEV 1

```
src/entities/product.ts
src/entities/category.ts
src/entities/tag.ts
src/repositories/product-repository.ts
src/repositories/category-repository.ts
src/components/products/
src/app/(store)/produtos/
src/app/admin/produtos/
src/app/admin/categorias/
```

DEV 2

```
src/entities/user.ts
src/entities/address.ts
src/entities/cart.ts
src/entities/order.ts
src/entities/payment.ts
src/entities/delivery.ts
src/repositories/cart-repository.ts
src/repositories/order-repository.ts
src/components/cart/
src/components/checkout/
```

Arquivos que exigem coordenação

Evitar edição simultânea para reduzir conflitos de merge:

```
package.json
src/app/globals.css
src/app/layout.tsx
src/storage/local-storage.ts
src/storage/seed.ts
```

Fluxo Git

- Uma branch por funcionalidade.
- Pull requests pequenos.
- Atualização frequente com a principal.
- Contratos integrados antes dos consumidores.
- Revisão cruzada entre os desenvolvedores.

Exemplos de branches

```
feature/catalog-domain
feature/product-catalog
feature/admin-products
feature/cart-domain
feature/checkout
feature/customer-orders
```

Resumo da divisão

DESENVOLVEDOR 1

Catálogo, produtos, estoque e administração do catálogo.

DESENVOLVEDOR 2

Cliente, carrinho, checkout, pagamento e pedidos.